

SQL Window Functions

Window functions

Window functions

- 1.Value
- 2.Ranking
- 3.Aggregate

1.Value

FIRST_VALUE()
LAST_VALUE()
LEAD()
LAG()

2.Ranking

DENSE_RANK()
RANK()
ROW_NUMBER()
PERCENT_RANK()
CUMMULATIVE SUM

3.Aggregate

SUM()
AVG()
MIN()
MAX()
COUNT()

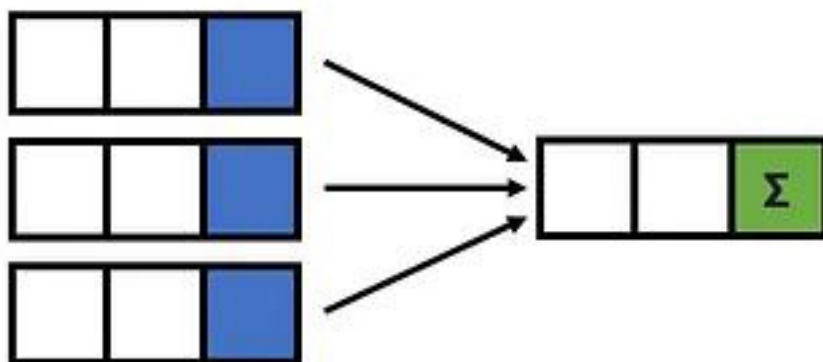
An Easy Guide to Advanced SQL Window Functions

- Window functions are an advanced type of function in SQL. They let you work with observations more easily.
- Window functions give you access to features like advanced analytics and data manipulation without the need to write complex queries.

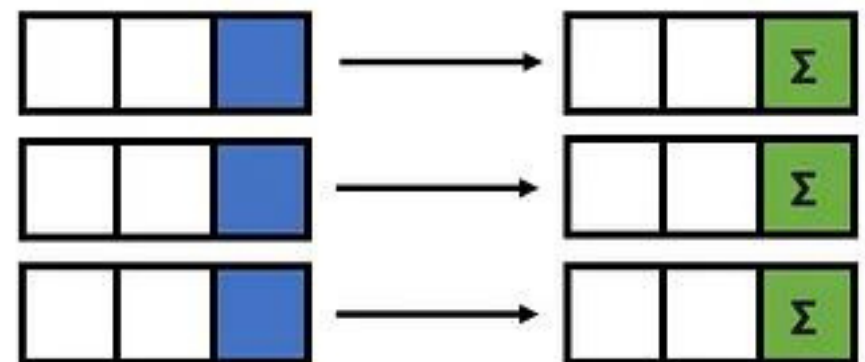
What is a Window Function?

- “A window function performs a calculation across a set of table rows that are somehow related to the current row...Behind the scenes, the window function is able to access more than just the current row of the query result.”
- Window functions are similar to the aggregation done in the GROUP BY clause. However, rows are not grouped into a single row, each row retains their separate identity.
- Window function may return a single value for each row. Here's a good visualization of what I mean by that.

Aggregate Functions (SUM, AVG, etc.)



Window Functions



Different Types of Window Functions

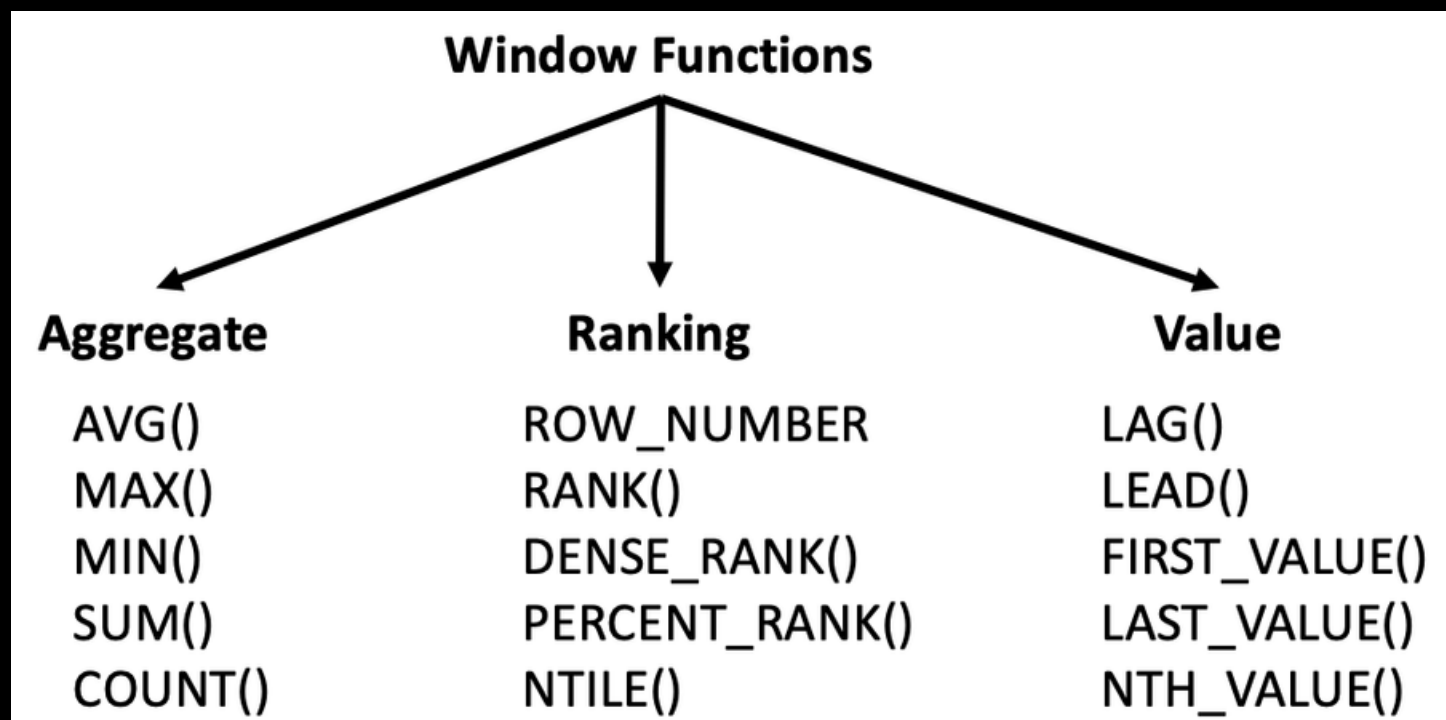
There are a lot of window functions that exist in SQL but they are primarily categorized into 3 different types:

Aggregate window functions

Value window functions

Ranking window functions

- Aggregate window functions are used to perform operations on sets of rows in a window(s). They include SUM(), MAX(), COUNT(), and others.
- Rank window functions are used to rank rows in a window(s). They include RANK(), DENSE_RANK(), ROW_NUMBER(), and others.
- Value window functions are like aggregate window functions that perform multiple operations in a window, but they're different from aggregate functions. They include things like LAG(), LEAD(), FIRST_VALUE(), and others.



Syntax for Window Functions

In a simple expression, a window function looks like this:

```
SELECT <column_1>, <column_2>,  
      <window_function> (expression) OVER  
      (PARTITION BY <partition_list> ORDER BY <order_list> ROWS frame_clause)  
FROM <table_name>
```

There's a lot of words here, so let's look at some definitions:

- **window_function** is the name of the window function we want to use; for example, sum, avg, or row_number (we'll learn more about these later)
- **expression** is the name of the column that we want the window function operated on. This may not be necessary depending on what window_function is used
- **OVER** is just to signify that this is a window function
- **PARTITION BY** divides the rows into partitions so we can specify which rows to use to compute the window function
- **partition_list** is the name of the column(s) we want to partition by
- **ORDER BY** is used so that we can order the rows within each partition. This is optional and does not have to be specified
- **order_list** is the name of the column(s) we want to order by
- **ROWS** can be used if we want to further limit the rows within our partition. This is optional and usually not used
- **frame_clause** defines how much to offset from our current row

Sample Table

In this tutorial you will be working with a table called `student_score` which contains data such as `student_id`, `student_name`, `dep_name` and `score`.

You can create the table using the following code:

```
CREATE TABLE student_score (  
  student_id SERIAL PRIMARY KEY,  
  student_name VARCHAR(30),  
  dep_name VARCHAR(40),  
  score INT  
);
```

```
INSERT INTO student_score VALUES (11, 'Ibrahim', 'Computer Science', 80);  
INSERT INTO student_score VALUES (7, 'Taiwo', 'Microbiology', 76);  
INSERT INTO student_score VALUES (9, 'Nurain', 'Biochemistry', 80);  
INSERT INTO student_score VALUES (8, 'Joel', 'Computer Science', 90);  
INSERT INTO student_score VALUES (10, 'Mustapha', 'Industrial Chemistry',  
78);  
INSERT INTO student_score VALUES (5, 'Muritadoh', 'Biochemistry', 85);  
INSERT INTO student_score VALUES (2, 'Yusuf', 'Biochemistry', 70);  
INSERT INTO student_score VALUES (3, 'Habeebah', 'Microbiology', 80);  
INSERT INTO student_score VALUES (1, 'Tomiwa', 'Microbiology', 65);  
INSERT INTO student_score VALUES (4, 'Gbadebo', 'Computer Science', 80);  
INSERT INTO student_score VALUES (12, 'Tolu', 'Computer Science', 67);
```

How to Use a Window Function – Example

Let's go over an example of how to use a window function. Say for instance you want to compare the minimum score and maximum score from all the records in the table we created earlier. You can do that using a window function as shown below.

Remember that not specifying a partition clause in the OVER clause will cause all the windows to span through the entire dataset.

```
SELECT
    *,
    MAX(score) OVER() AS maximum_score,
    MIN(score) OVER() AS minimum_score
```

```
FROM student_score;
```

As you can see, we have the minimum and maximum salary across the entire dataset.

	student_id [PK] integer	student_name character varying (30)	dep_name character varying (40)	score integer	maximum_score integer	minimum_score integer
1	11	Ibrahim	Computer Science	80	90	65
2	7	Taiwo	Microbiology	76	90	65
3	9	Nurain	Biochemistry	80	90	65
4	8	Joel	Computer Science	90	90	65
5	10	Mustapha	Industrial Chemistry	78	90	65
6	5	Muritadoh	Biochemistry	85	90	65
7	2	Yusuf	Biochemistry	70	90	65
8	3	Habeebah	Microbiology	80	90	65
9	1	Tomiwa	Microbiology	65	90	65
10	4	Gbadebo	Computer Science	80	90	65

Total rows: 11 of 11 | Query complete 00:00:00.004

How to Use a Window Function with PARTITION BY

- Say, for instance, that you want to split the dataset into different partitions. Then you want to compare each record in each partition with an aggregate value or a calculated value of each partition. You can specify the PARTITION BY clause in the OVER function.
- For example, say you want to compare the maximum score and average score in each department with the individual score. You can do this by specifying the PARTITION BY clause in the OVER statement and also use it with the aggregate function you want to use to achieve your desired result.

```
SELECT  
    *,  
    MAX(score)OVER(PARTITION BY dep_name) AS dep_maximum_score,  
    ROUND(AVG(score)OVER(PARTITION BY dep_name), 2) AS dep_average_score  
FROM student_score;
```

You can see that the PARTITION BY clause specified in the OVER() clause split the result set into 4 different partitions. This is because there are 4 different departments in the dep_name column (which are Biochemistry, Computer Science, Industrial Chemistry, and Microbiology).

Now after the PARTITION BY clause, you can then calculate the aggregate function for each record in the different departments.

	student_id [PK] integer	student_name character varying (30)	dep_name character varying (40)	score integer	dep_maximum_score integer	dep_average_score numeric
1	2	Yusuf	Biochemistry	70	85	78.33
2	5	Muritadoh	Biochemistry	85	85	78.33
3	9	Nurain	Biochemistry	80	85	78.33
4	12	Tolu	Computer Science	67	90	79.25
5	8	Joel	Computer Science	90	90	79.25
6	4	Gbadebo	Computer Science	80	90	79.25
7	11	Ibrahim	Computer Science	80	90	79.25
8	10	Mustapha	Industrial Chemistry	78	78	78.00
9	3	Habeebah	Microbiology	80	80	73.67
10	1	Tomiwa	Microbiology	65	80	73.67

Total rows: 11 of 11 Query complete 00:00:00.124 Ln 5, Col 20

How to Use the ROW_NUMBER Function

- ROW_NUMBER() to assign serial numbers to records in a window. Say we want to assign serial numbers to the records in a partition.
- For example, we want to add row numbers to the dataset based on their names in alphabetical order. You can do that using the following code:

SELECT

*****,

ROW_NUMBER() OVER(ORDER BY student_name) AS name_serial_number
FROM student_score;

	student_id [PK] integer	student_name character varying (30)	dep_name character varying (40)	score integer	name_serial_number bigint
1	4	Gbadebo	Computer Science	80	1
2	3	Habeebah	Microbiology	80	2
3	11	Ibrahim	Computer Science	80	3
4	8	Joel	Computer Science	90	4
5	5	Muritadoh	Biochemistry	85	5
6	10	Mustapha	Industrial Chemistry	78	6
7	9	Nurain	Biochemistry	80	7
8	7	Taiwo	Microbiology	76	8
9	12	Tolu	Computer Science	67	9
10	1	Tomiwa	Microbiology	65	10

As you can see from the above image, the student_name with the smallest value (that is, the one that falls earliest in the alphabet) is Gbadebo since it starts with G. Then 1 is added as its row number which is followed by the name that begins with H, and so on.

How to Use the RANK Function

- RANK(), as the name implies, lets you rank observations in a window but with gaps. Let's see what this means:

SELECT

*****,

RANK()OVER(PARTITION BY dep_name ORDER BY score DESC)

FROM student_score;

	student_id [PK] integer	student_name character varying (30)	dep_name character varying (40)	score integer	rank bigint
1	5	Muritadoh	Biochemistry	85	1
2	9	Nurain	Biochemistry	80	2
3	2	Yusuf	Biochemistry	70	3
4	8	Joel	Computer Science	90	1
5	11	Ibrahim	Computer Science	80	2
6	4	Gbadebo	Computer Science	80	2
7	12	Tolu	Computer Science	67	4
8	10	Mustapha	Industrial Chemistry	78	1
9	3	Habeebah	Microbiology	80	1
10	7	Taiwo	Microbiology	76	2

- As you can see in the above code, the result set was partitioned into different windows based on the department column. Then we used the ORDER BY clause to sort the student records based on their score in descending order in each partition. After that, we applied the RANK function.
- Now concerning the gaps, as we can see in the highlighted part in the above image, two records in the Computer Science department have the same score (80). This caused both to be ranked with the value 2 (instead of one being ranked 2 and the other 3). So it doesn't know how to handle a tie, basically.
- We can avoid this scenario using another window function called DENSE_RANK that ranks observations in a window without these gaps.

How to Use the DENSE_RANK Function

- **DENSE_RANK** is similar to RANK except that it ranks observations in a window without gaps.

SELECT

*,

DENSE_RANK()OVER(PARTITION BY dep_name ORDER BY score DESC)
FROM student_score;

	student_id [PK] integer	student_name character varying (30)	dep_name character varying (40)	score integer	dense_rank bigint
1	5	Muritadoh	Biochemistry	85	1
2	9	Nurain	Biochemistry	80	2
3	2	Yusuf	Biochemistry	70	3
4	8	Joel	Computer Science	90	1
5	11	Ibrahim	Computer Science	80	2
6	4	Gbadebo	Computer Science	80	2
7	12	Tolu	Computer Science	67	3
8	10	Mustapha	Industrial Chemistry	78	1
9	3	Habeebah	Microbiology	80	1
10	7	Taiwo	Microbiology	76	2

As we can see in the output above, when using DENSE_RANK, the next rank number (which is 3) was assigned to Tolu (unlike when using RANK which assigned Tolu a rank of 4, skipping 3 because of the tie).

How to Use the LAG Function

- LAG is used to return the offset row before the current row within a window. By default it returns the previous row before the current row.
- You typically use LAG when you want to compare the value of a previous row with the current row. It's commonly applied in time-series analysis.
- For example:

SELECT

*****,

LAG(score) OVER(PARTITION BY dep_name ORDER BY score)

FROM student_score;

	student_id [PK] integer	student_name character varying (30)	dep_name character varying (40)	score integer	lag integer
1	2	Yusuf	Biochemistry	70	[null]
2	5	Muritadoh	Biochemistry	85	70
3	9	Nurain	Biochemistry	80	85
4	12	Tolu	Computer Science	67	[null]
5	8	Joel	Computer Science	90	67
6	4	Gbadebo	Computer Science	80	90
7	11	Ibrahim	Computer Science	80	80
8	10	Mustapha	Industrial Chemistry	78	[null]
9	3	Habeebah	Microbiology	80	[null]
10	1	Tomiwa	Microbiology	65	80

As shown in the first partition, the first record in the biochemistry partition (Yusuf's) does not have a previous value (that is, no record comes before it) so that's why null was returned. Then moving to the next record – Muritadoh's – it has a previous record, so it returns the previous value which is 70.

How to Use the Frame Clause in ORDER BY

- Now you've learned some common window functions you might work with on a daily basis. So let's move on to learning another key concept related to the ORDER BY clause called the frame clause.
- A frame clause, as the name implies, provides the frame (that is, the set of rows in a window) on which the function is to be applied. You use it to provide the offset of rows to be included or calculated with the current row (that is, the rows before or after the current row – the SQL engine process row one after the other).
- Now before we look into how to specify a frame clause, let's look at some of the frame clause's assumptions:
- First, a frame clause does not apply to ranking functions. The ranking function only ranks the observation in the window based on the ORDER BY clause.
- When using an aggregate window function, you may not include the ORDER BY clause. But when you use the ORDER BY clause, it's a best practice to specify the frame clause for accurate results. What this means is say you want to use an aggregate window function and you want to also order the observations in that window by a column. It's best practice is to specify a frame clause so that you will get an accurate result. But if you are not ordering the observations in the window when using an aggregate function, you don't need to specify a frame clause.
- We can specify a frame clause using two things – ROWS and RANGE. But in this part you will learn how to use the ROWS keyword since it is commonly used to specify a frame clause.
- The ROWS clause defines the frame in terms of the physical offset rows from the current rows. That is, it is used to specify the rows that will be used in conjunction with the current row for calculation.

For example the following frame clause **ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING** defines a frame that includes the current row, 1 row preceding it and 1 row following it.

Let's look at the keywords that you can use in conjunction with the **ROWS** clause:

- **N PRECEDING** is a keyword you use to specify the **N** rows that will be included in the calculation along with the current row. For example **3 PRECEDING** means 3 rows preceding the current row.
- **N FOLLOWING** works like **N PRECEDING** excepts that it works in an opposite manner. **N FOLLOWING** specifies the numbers of row after the current row.
- **UNBOUNDED PRECEDING** means all rows before the current row.
- **UNBOUNDED FOLLOWING** means all rows after the current row.
- **CURRENT ROW** is used to specify the current row.

For example, let's look at the below frame clause:

ROWS BETWEEN 2 PRECEDING AND CURRENT ROW will use less than or equal to 2 rows before the current row, along with the current row for the calculation.

Frame clause example

- Let's look at an example. Say for instance you want to get the cumulative sum of all the student scores. You can do that by using a frame clause.
- So first, to be able to do this, you need to first know the types of keywords you will specify in the frame clause.
- Since you want to sum up all rows before the current row and the current row itself, you can use the UNBOUNDED PRECEDING keyword. Remember that this gets all rows before the current row and also uses the current row itself.
- So the code to achieve that task is shown below:

SELECT

,

**SUM(score)OVER(ORDER BY student_id ROWS BETWEEN UNBOUNDED
PRECEDING AND CURRENT ROW) AS cummulative_sum
FROM student_score**

Let's break down the window function code:

- SUM(score)OVER(ORDER BY student_id ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS cummulative_sum
- Firstly in the OVER() clause, we sort the entire window – which is the whole dataset – using the student id.
- Then we specify the frame clause which is ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW. This is all rows before the current row and the current row will be used for calculation.

The result is shown in the below image:

	student_id [PK] integer	student_name character varying (30)	dep_name character varying (40)	score numeric	cummulative_sum numeric
1	1	Tomiwa	Microbiology	65	65
2	2	Yusuf	Biochemistry	70	135
3	3	Habeebah	Microbiology	80	215
4	4	Gbadebo	Computer Science	80	295
5	5	Muritadoh	Biochemistry	85	380
6	7	Taiwo	Microbiology	76	456
7	8	Joel	Computer Science	90	546
8	9	Nurain	Biochemistry	80	626
9	10	Mustapha	Industrial Chemistry	78	704
10	11	Ibrahim	Computer Science	80	784

- The first row in the dataset does not have any row before it. But since we also specify the CURRENT ROW keyword as the last frame, then the SQL engine finds its sum which equals 65.
- Then moving to the second row. It has 1 row before it. So the SQL engine sums the score of the first row 65 with the current row which is 70. That is why the result is 135...and so on down the table.

When to Use a Window Function

- When you want to compare an aggregate value in a window with individual records in that window.
- When you want to do things like ranking, percentile, cumulative sum or running total, moving average, and so on.

Conclusion

- We've learned what window functions are, and we've also looked at some of the clauses we can add in Windows functions. One example is the PARTITION BY clause, which divides the result set into separate partitions or windows.
- We also learned how to utilize the ORDER BY clause to order observations in a window and we saw various common examples of window functions.
- Finally, we learned another advanced clause that we can use with window functions, the frame clause, which allows us to access more features of a window.

What Are Your Takeaway From This Post?

Let Me Know in Comment Section
and Do Share This Post With Your
Network So That Everyone Gets
Benefited



Follow